# Experiment-09: Travelling Salesman Problem (TSP) using Python

**Aim**

To develop a Python program to solve the **Travelling Salesman Problem (TSP)** using the **Brute Force** approach by finding the minimum-cost tour that visits every city exactly once and returns to the starting city.

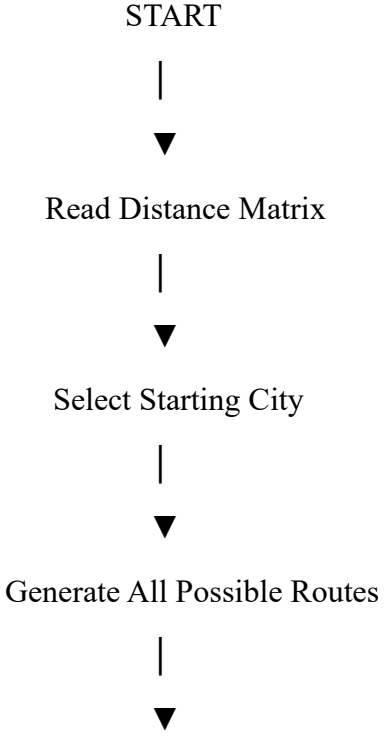
---

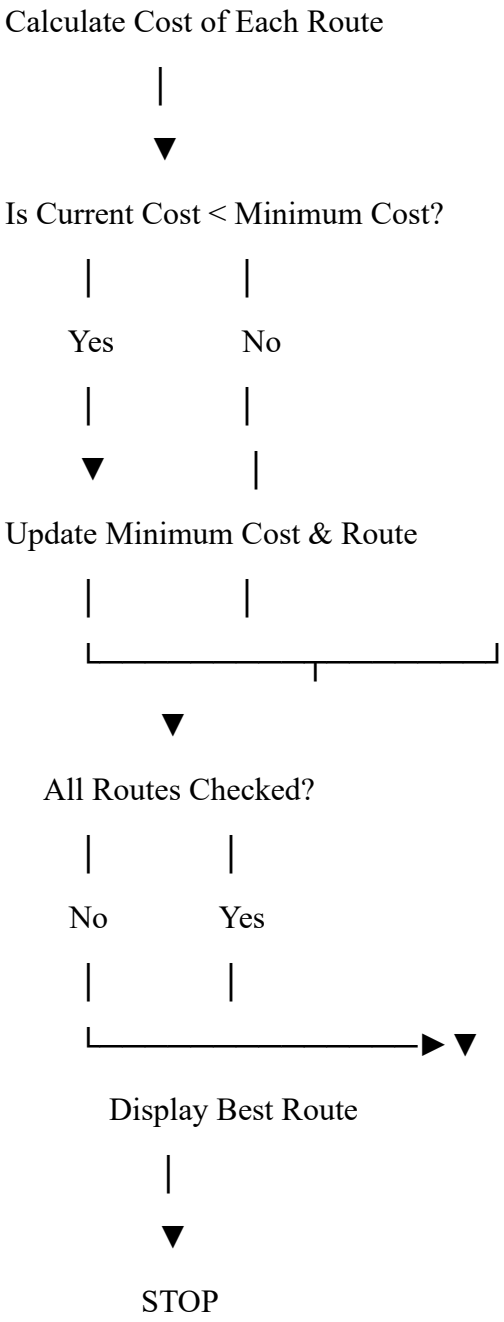
**Algorithm**

1. Represent the cities and distances using a distance matrix.
2. Select the starting city.
3. Generate all possible permutations of the remaining cities.
4. Calculate the total travel cost for each possible route.
5. Compare the costs and store the minimum-cost route.
6. Display the optimal route and its minimum cost.
7. Stop.

---

**Flowchart**






---

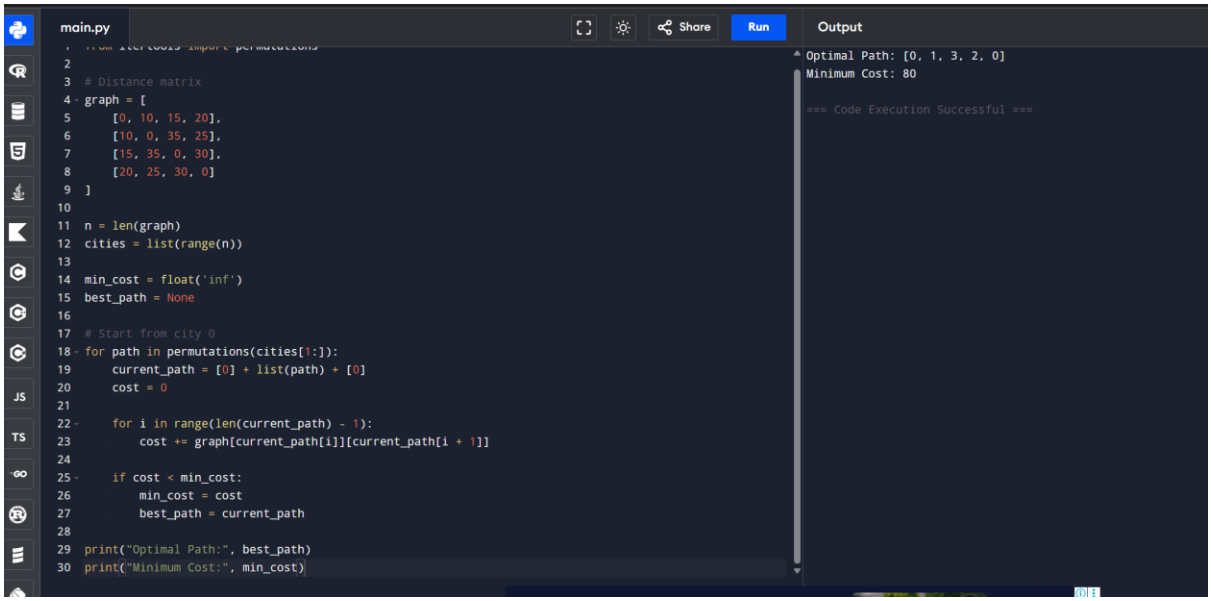
**Python Code**

```
from itertools import permutations


# Distance matrix
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
```

```
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
cities = list(range(n))

min_cost = float('inf')
best_path = None

# Start from city 0
for path in permutations(cities[1:]):
    current_path = [0] + list(path) + [0]
    cost = 0

    for i in range(len(current_path) - 1):
        cost += graph[current_path[i]][current_path[i + 1]]

    if cost < min_cost:
        min_cost = cost
        best_path = current_path

print("Optimal Path:", best_path)
print("Minimum Cost:", min_cost)
```

---

**Output**




---

### Result

The **Travelling Salesman Problem (TSP)** was successfully implemented in Python using the **Brute Force** approach. The program generated all possible tours, calculated the total travel cost for each route, and identified the **minimum-cost tour**.

- **Optimal Path:** 0 → 1 → 3 → 2 → 0
- **Minimum Cost: 80**